

Estrategias para construir algoritmos

Estructura de Datos y Algoritmos I

Profesora: Karen Sáenz

Profesora: Elba Karen Sáenz García



¿Qué es una técnica de diseño de algoritmos?

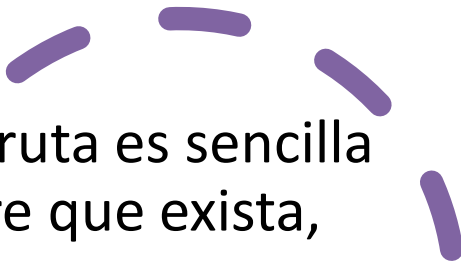
- Llamada también estrategia o paradigma
- Es una forma o estrategia para resolver un problema utilizando un algoritmo que es aplicable a una variedad de problemas de diferentes áreas de la computación.

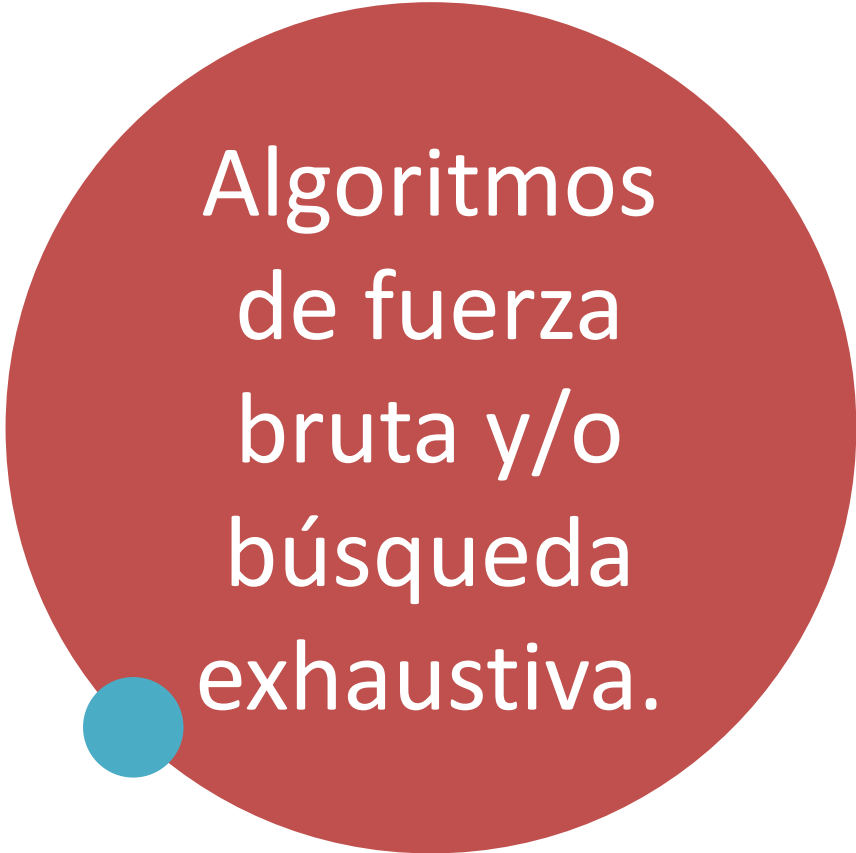
Algoritmos de fuerza bruta y/o búsqueda exhaustiva.

- Técnica trivial y muy utilizada
- Consiste en enumerar sistemáticamente todos los posibles candidatos para la solución de un problema, con el fin de ir revisando si dicho candidato satisface la solución al mismo(problema).



Algoritmos de Fuerza bruta.

- 
- La búsqueda por fuerza bruta es sencilla de implementar y, siempre que exista, encuentra una solución.
 - Sin embargo, su coste de ejecución es proporcional al número de soluciones candidatas, el cual es exponencialmente proporcional al tamaño del problema.



Algoritmos de fuerza bruta y/o búsqueda exhaustiva.

- Utilizado cuando es **más importante** una implementación sencilla que una mayor rapidez.
- Este puede ser el caso en aplicaciones críticas donde cualquier error en el algoritmo puede acarrear serias consecuencias

Ejemplo1

- Un algoritmo de fuerza bruta para encontrar el divisor de un numero natural n consistiría en enumerar todos los enteros desde 1 hasta n , revisando si cada uno de ellos divide n sin generar residuo.

¿Cómo?





Ejemplo2

- Problema de las 8 reinas: posicionar ocho reinas en el tablero de ajedrez de forma que ninguna de ellas ataque al resto.
- Solución: Examinar todas las combinaciones de posición para las 8 reinas (en total $64! / 56! = 178.462.987.637.760$ posiciones diferentes), comprobando en cada una de ellas si las reinas se atacan mutuamente.



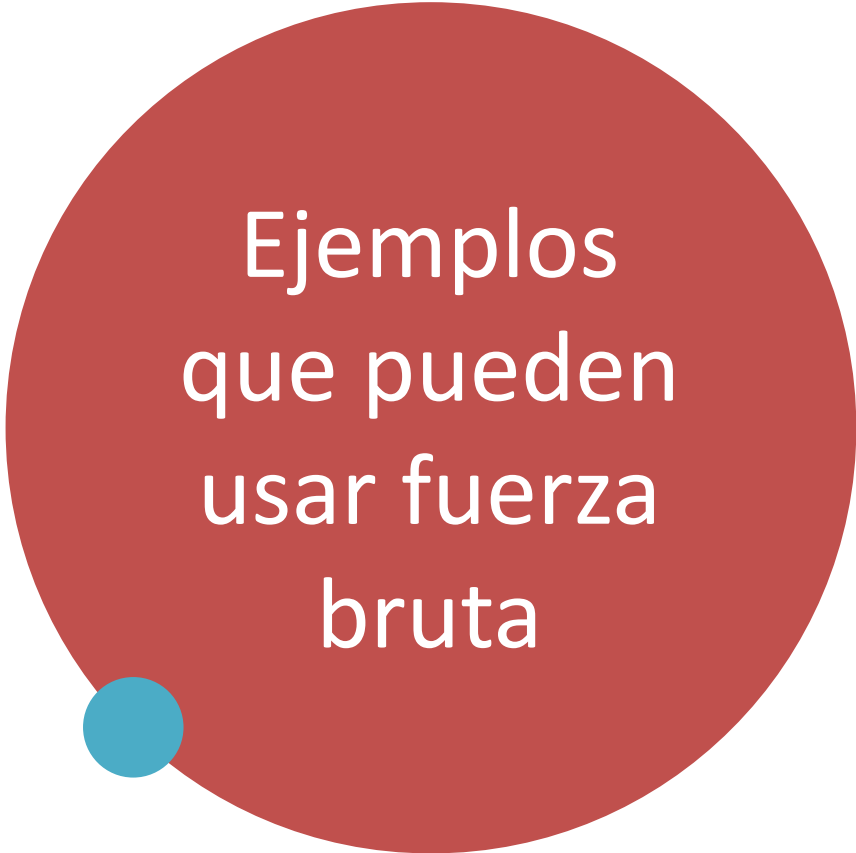
Algoritmo General

- Implementar las funciones:
 1. **primero (P)**: genera la primera solución candidata para P.
 2. **siguiente (P, c)**: genera la siguiente solución candidata para P después de una solución candidata c.
 3. **valido (P, c)**: revisa si una solución candidata c es una solución correcta de P.
 4. **mostrar (P, c)**: informa que la solución c es una solución correcta de P.



Un Algoritmo General

```
c = primero(P)
Mientras c != null
    Si valido(P,c) entonces
        mostrar(P, c)
    Fin Si
    c = siguiente(p,c)
finMientras
```

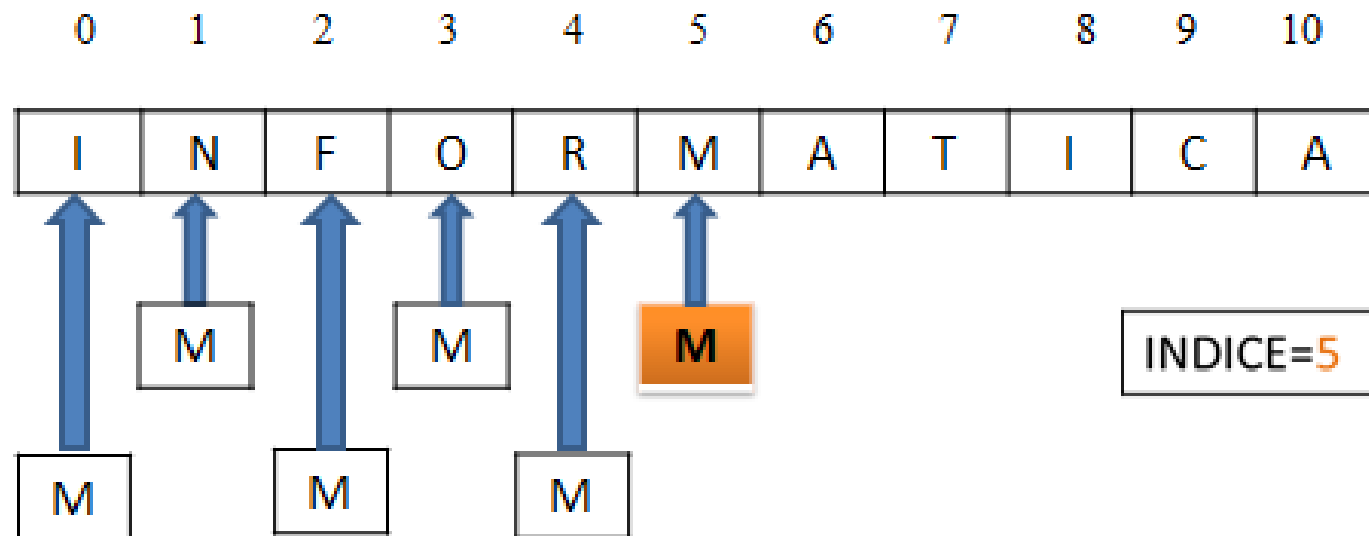


Ejemplos que pueden usar fuerza bruta

- Búsqueda lineal iterativa
 - Ordenación por selección
 - Closest-Pair Problem
 - N reinas
- 

Algoritmo Búsqueda Secuencial o Lineal

- Si se busca la llave x
- Se empieza a recorrer el arreglo desde la primera posición de la lista, comparando el elemento con la llave x, si no se encuentra continúa verificando las siguientes posiciones hasta encontrarlo o terminar de recorrerlo, entonces el algoritmo termina.



Algoritmo búsqueda lineal

BúsquedaLineal (A,n,x)

Inicio

 encontrado=-1

 Para k=0 hasta n-1

 Si $x == A[k]$

 encontrado = k

 Fin Si

 Fin Para

 Retorna encontrado

Fin

- Entrada:

A arreglo o lista

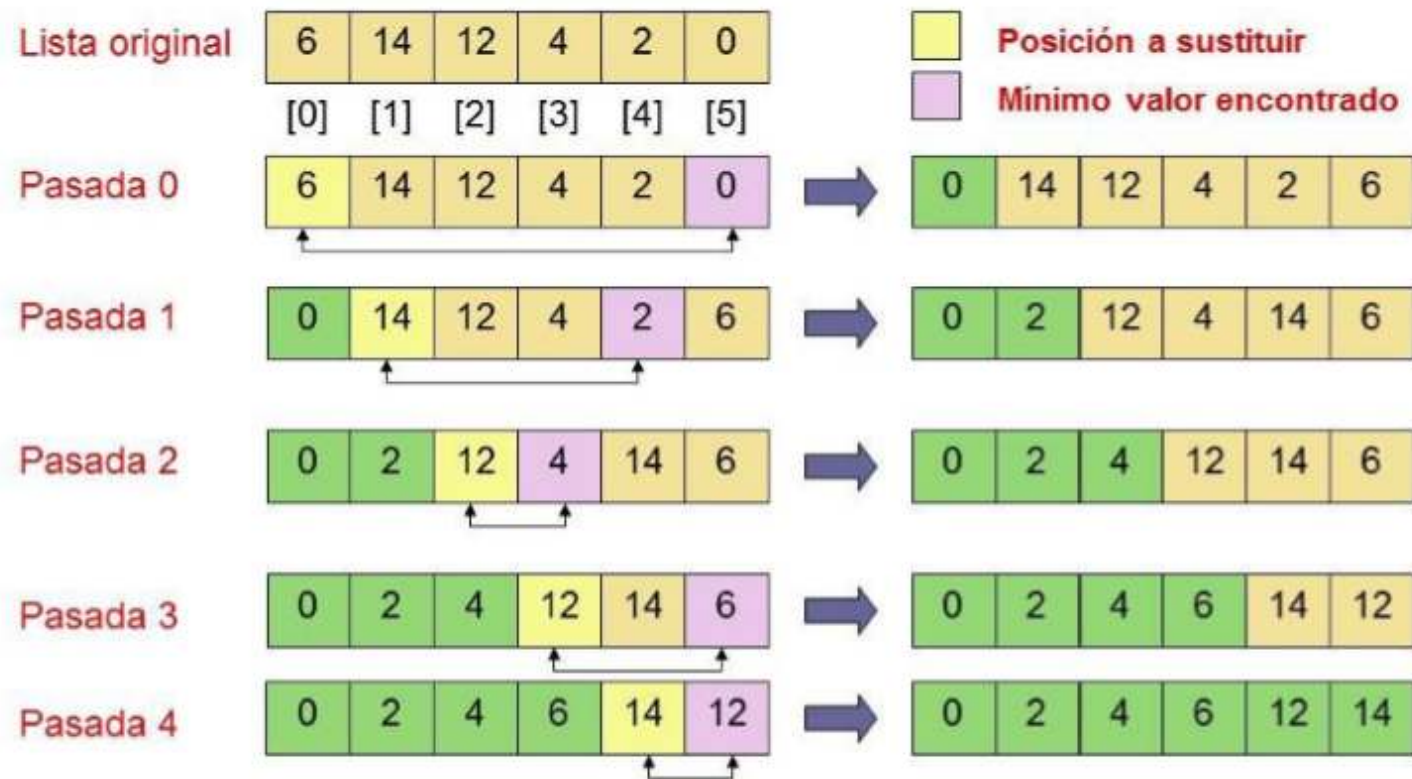
n numero de elementos

x llave (valor a buscar)

Solución Python

- Solución en clase

Ordenación por selección





Algoritmo

ordenacionPorSeleccion(A)

Inicio

Desde i=0 hasta n-1 hacer

min=i;

Desde j=i+1 hasta n-1 hacer

Si $A[j] < A[\text{min}]$ entonces

min=j

fin si

Fin Desde

Intercambia(A,i,min)

FinDesde

Fin ordenacionPor Seleccion

Ejemplo en Python

Closest-Pair Problem

- Caso dos dimensiones, entrada una lista de puntos $p(x_i, y_i)$

$$d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}.$$

- Calcular la distancia entre cada par de puntos distintos, encontrar el par con la distancia mas pequeña.
- Para no calcular la distancia entre dos puntos dos veces se consideran los puntos (p_i, p_j) para los cuales $i < j$

Algoritmo

- Entrada: Una lista P de n puntos ($n \geq 2$) $P_0(x_0, y_0), P_1(x_1, y_1), P_2(x_2, y_2), \dots, P_{n-1}(x_{n-1}, y_{n-1})$
- Salida: La distancia entre los pares de puntos más cercanos.

ClosestPair(P)

Inicio

dmin= se inicializa en un valor relativamente grande

Para i=0 hasta n-1

Para j=i+1 hasta n-1

$$d = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

Si $d < \text{dmin}$

dmin=d

p1=i

p2=j

Fin Si

Fin Para

Fin Para

Retornar d

Fin

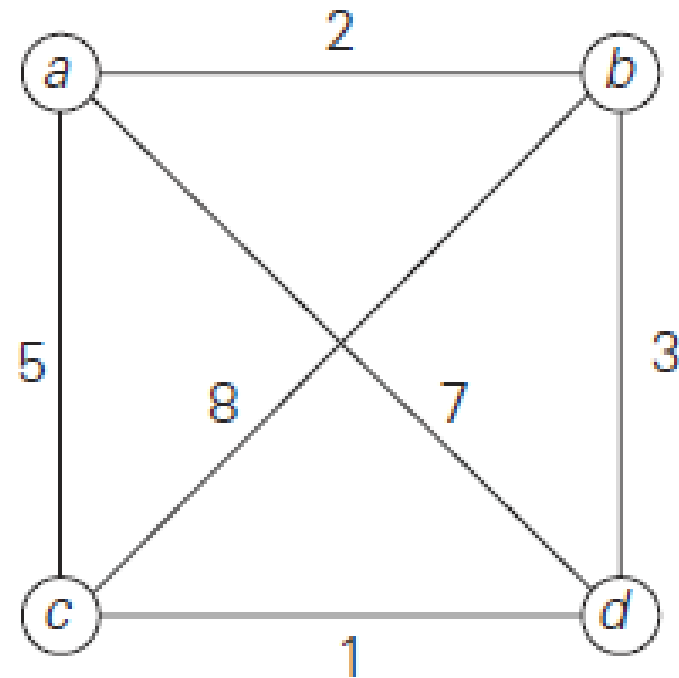
Ejemplo en Python

Búsqueda Exhaustiva

- Fuerza bruta en para problemas donde se busca un elemento con una propiedad especial, usualmente entre objetos combinatorios como permutaciones, combinaciones, o subconjuntos (problemas combinatoriales)
-
- Muchos de los problemas son de optimización.

Traveling Salesman Problem

Dada una lista de n ciudades y las distancias entre cada par de ellas, encontrar el recorrido (ruta) más corto posible que visita cada ciudad exactamente una vez y regresa a la ciudad origen.



Traveling Salesman Problem

<u>Tour</u>	<u>Length</u>	
$a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$	$l = 2 + 8 + 1 + 7 = 18$	
$a \rightarrow b \rightarrow d \rightarrow c \rightarrow a$	$l = 2 + 3 + 1 + 5 = 11$	optimal
$a \rightarrow c \rightarrow b \rightarrow d \rightarrow a$	$l = 5 + 8 + 3 + 7 = 23$	
$a \rightarrow c \rightarrow d \rightarrow b \rightarrow a$	$l = 5 + 1 + 3 + 2 = 11$	optimal
$a \rightarrow d \rightarrow b \rightarrow c \rightarrow a$	$l = 7 + 3 + 8 + 5 = 23$	
$a \rightarrow d \rightarrow c \rightarrow b \rightarrow a$	$l = 7 + 1 + 8 + 2 = 18$	

- Para n ciudades hay $n!$ soluciones potenciales (número de permutaciones de n elementos)
- En varias de ellas sólo cambia el sentido del recorrido, por lo que existen $n! / 2$ posibles soluciones distintas El algoritmo de búsqueda exhaustiva pertenece entonces a la clase $O(n!)$

Problema de la Mochila

Se tienen n distintos tipos de objetos, cada tipo de objeto i tiene un beneficio o valor asociado v_i y un peso (o volumen) w_i ($v_i, w_i > 0$).

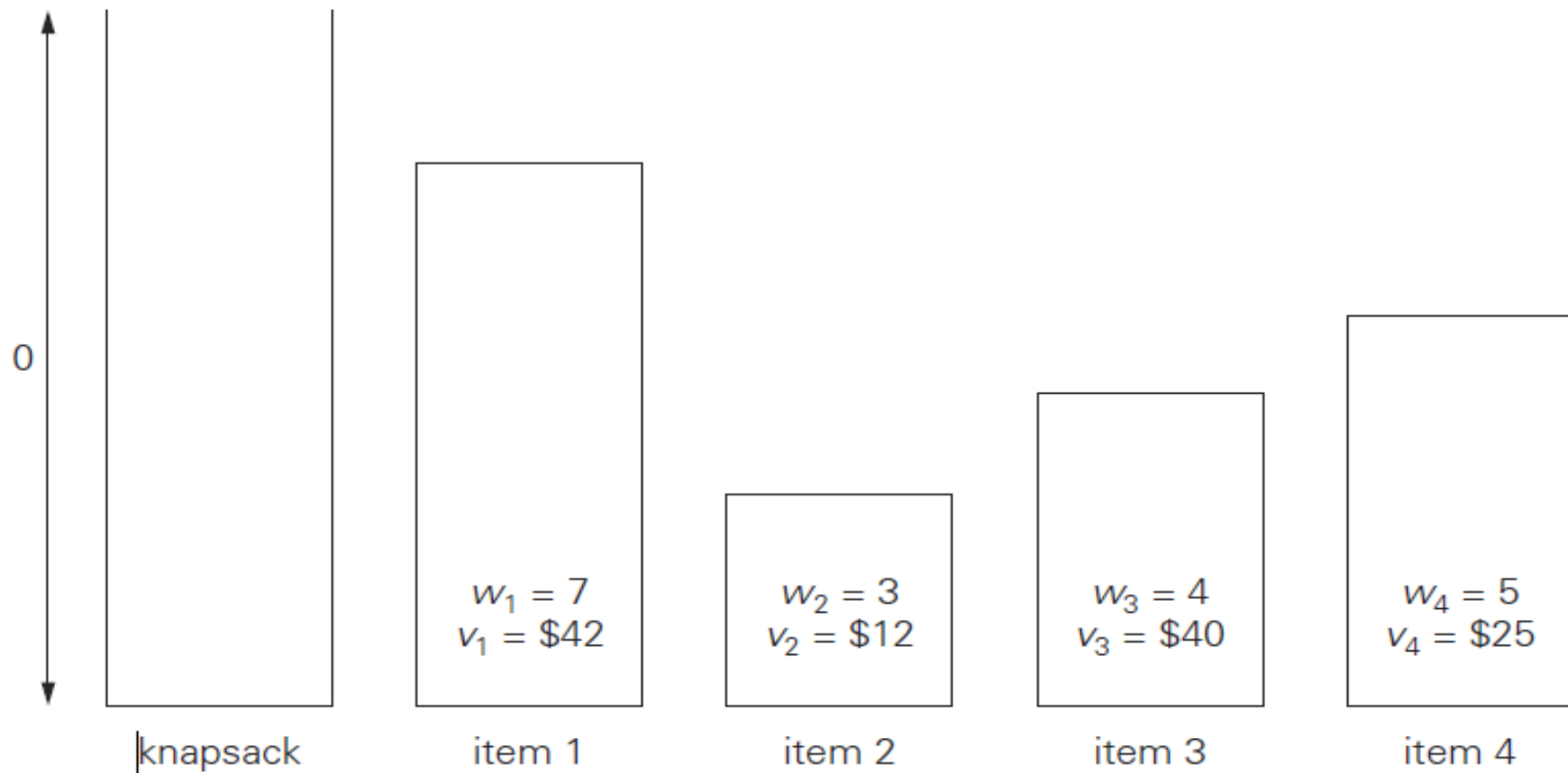
Se tiene una mochila, donde se pueden introducir los objetos, que soporta un peso máximo (o volumen máximo) W .

Meter en la mochila objetos de tal forma que se maximice el valor de los objetos que contiene y siempre que no se supere el peso máximo que puede soportar la misma.

La solución será la secuencia de variables $x_1, x_2, \dots, x_n$ donde el valor de x_i indica el objeto que se meterá



Problema de la mochila



Subset	Total weight	Total value
$\emptyset$	0	\$ 0
{1}	7	\$42
{2}	3	\$12
{3}	4	\$40
{4}	5	\$25
{1, 2}	10	\$54
{1, 3}	11	not feasible
{1, 4}	12	not feasible
{2, 3}	7	\$52
{2, 4}	8	\$37
{3, 4}	9	\$65
{1, 2, 3}	14	not feasible
{1, 2, 4}	15	not feasible
{1, 3, 4}	16	not feasible
{2, 3, 4}	12	not feasible
{1, 2, 3, 4}	19	not feasible

Problema de la mochila

- El problema se puede resolver con búsqueda exhaustiva generando todos los subconjuntos del conjunto de n objetos (2^n), y evaluando cada uno de ellos para encontrar el mejor
- Por lo tanto la búsqueda exhaustiva nos conduce a un algoritmo con complejidad $\Omega(2^n)$